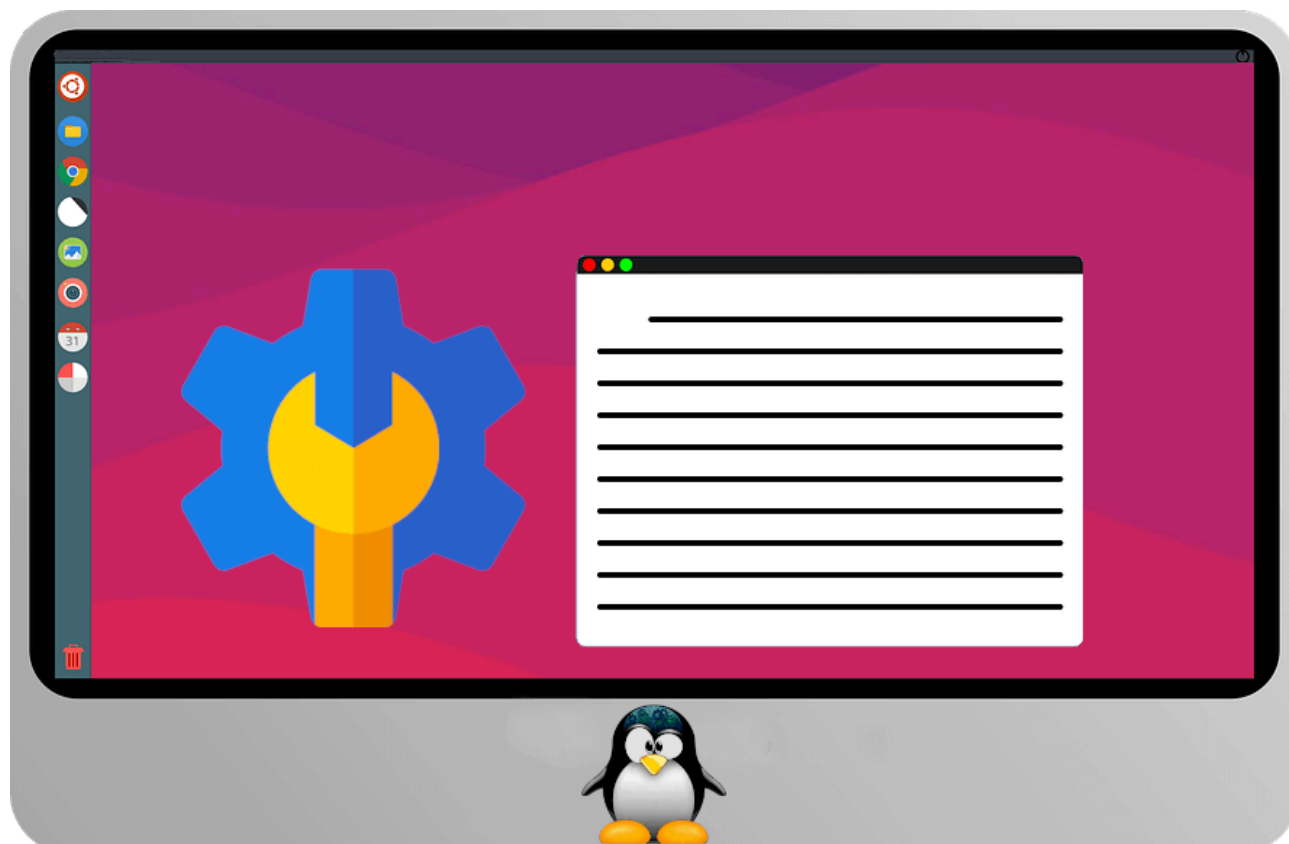


Secure LUKS containers on Linux – Michael Waterman

 michaelwaterman.nl/2025/10/14/secure-luks-container-on-linux

Michael Waterman

October 14, 2025



Lately, I've been refining parts of my Linux workflow to make them both more secure and practical. One of those improvements came from something simple but powerful, using encrypted containers instead of relying solely on full-disk encryption.

In this post, I'll walk you through how I built a secure, self-contained LUKS container on Linux, explain what's really happening behind the scenes, and share a few lessons learned along the way, including troubleshooting tips and two small Bash scripts that make mounting and unmounting effortless.

Over the years, I've worked with many forms of encryption, from full disk encryption on Windows with BitLocker to enterprise PKI deployments and hardware-backed FIDO2 authentication. But sometimes, all you need is something simpler: a secure, isolated container that behaves like a regular drive but is completely encrypted when closed.

That's where LUKS (*Linux Unified Key Setup*) shines. It's flexible, reliable, and built right into Linux. In this post, I'll walk through how I set up a LUKS container, what actually happens under the hood, and how to manage it securely with a few handy scripts.

Why use a LUKS container anyway?

A LUKS container is essentially a single encrypted file that acts like a virtual drive, it's as simple as that. When you open it, Linux maps it through a virtual block device, you can format it, mount it, and treat it like any other disk. When you close it, it's just an unreadable blob of encrypted data. It's perfect for:

- Isolating sensitive files in a secure vault.
- Testing encryption setups without touching your main drive.
- Portable or lab environments where you want full control.

And best of all, it doesn't require any external tools. Just `cryptsetup`, `losetup`, and a bit of Linux know-how, which I'll teach you in this post.

Prerequisites: Install cryptsetup

`cryptsetup` provides the user-space tools to create and manage LUKS volumes. On most modern Linux distributions it isn't installed by default, so let's add it first.

```
sudo apt update
sudo apt install -y cryptsetup
# Optional but handy for troubleshooting:
sudo apt install -y lsof
```

Verify the installation

If you're running a very minimal kernel build and run into errors, make sure the device-mapper `crypto` module is available:

```
lsmod | grep dm_crypt || sudo modprobe dm_crypt
```

Step-by-Step: Creating the encrypted container

Now, let's start by creating a 10 GB secure container called `secure_container.img`. It really does not matter what you name the file, it doesn't even need an extension. As long as it makes sense to you, you're good to go.

```
dd if=/dev/zero of=secure_container.img bs=1M count=10240
```

This command will create a 10 GB (10240 x 1 MB) container at the location you specify in the "of" parameter. The command above will create the container in the current working directory. Change the size to your needs via the count parameter. Please note that this process will take a while, just give it some time to complete!

Note! You might notice that the command above uses `/dev/zero` instead of `/dev/urandom`. Both can be used to create an empty image file, but the difference lies in what fills the space.

`/dev/zero` writes only null bytes (0x00). It's fast and efficient, which makes it ideal for creating the base file of a LUKS container. When you later run `cryptsetup luksFormat`, LUKS itself handles all cryptographic initialization and fills the necessary blocks with secure random data.

`/dev/urandom`, on the other hand, writes random bytes. This makes the container file look completely random from the outside, a benefit if you want *plausible deniability* or to obscure the fact that the file even contains an encrypted volume. The trade-off is performance: creating large files with `/dev/urandom` is significantly slower.

In my own workflow, I stick with `/dev/zero`. It keeps the setup fast and clean, and once LUKS takes over, every bit that truly matters is already encrypted and randomized. For me, that's the right balance between practicality and security.

Next, format it as a LUKS volume:

```
cryptsetup luksFormat secure_container.img
```

Type “YES” on the “are you sure?” question, type in the passphrase twice and let LUKS take care of the rest. Now we'll map it using a loop device:

```
sudo losetup -fP --show secure_container.img  
sudo cryptsetup luksOpen /dev/loop11 securecontainer
```

The parameters used here serve these purposes:

`-f`: Find the first unused loop device and use that one. `-P`: Create a partitioned loop device—`show`: Print the used device name on the screen.

We need the latter to use the correct loop device afterwards. If you need to lookup the name afterwards, there's the “`losetup -a`” to do just that.

```
superuser@X302LA:~$ losetup -a  
/dev/loop1: []: (/var/lib/snapd/snaps/core22_2111.snap)  
/dev/loop8: []: (/var/lib/snapd/snaps/gtk-common-themes_1535.snap)  
/dev/loop6: []: (/var/lib/snapd/snaps/snapd_24792.snap)  
/dev/loop4: []: (/var/lib/snapd/snaps/anome-42-2204_202.snap)  
/dev/loop11: []: (/home/superuser/secure_container.img)  
/dev/loop2: []: (/var/lib/snapd/snaps/core22_2133.snap)  
/dev/loop0: []: (/var/lib/snapd/snaps/bare_5.snap)
```

Now that we've created the virtual device `/dev/mapper/securecontainer`, we can format it with a filesystem (I usually prefer ext4):

```
sudo mkfs.ext4 /dev/mapper/securecontainer
```

And finally, mount it somewhere convenient:

```
mkdir -p ~/securedata  
sudo mount /dev/mapper/securecontainer ~/securedata
```

At this point, you have a fully functional encrypted container. Anything written to `~/securedata` is transparently encrypted within your `.img` file. Cool, or what!

The role of the mapper

When you open a LUKS container, something interesting happens behind the scenes. `cryptsetup` doesn't directly decrypt your data, instead, it leverages the “*Linux device mapper*” subsystem.

The mapper creates a “*virtual block device*” under `/dev/mapper/` that intercepts all reads and writes:

- When you write a file, the mapper encrypts the data block-by-block.
- When you read a file, it decrypts those blocks transparently.
- The encrypted data itself lives inside your `.img` file, accessed through the loop device.

This layered approach is one of Linux's greatest strengths. It allows encryption to coexist with other features like LVM, RAID, and even snapshots, all without any special drivers or kernel hacks.

Container management

When you're done working with the container, closing it cleanly is important to prevent data corruption within the container.

```
# Unmount the filesystem:  
sudo umount ~/securedata  
# Close the LUKS mapping:  
sudo cryptsetup luksClose securecontainer  
#Detach the loop device:  
sudo losetup -d /dev/loop11
```

Note! If you forget which mappings are open:

```
sudo cryptsetup status securecontainer
```

Troubleshooting common issues

“Device is busy” when unmounting

Sometimes a process is still using the mounted directory

```
sudo lsof /mnt/securedata
sudo fuser -m /mnt/securedata
```

Once you close those, you can safely unmount.

Enumerating active containers

cryptsetup doesn't have a literal “enumerate” command, but you can easily check active mappings:

```
ls /dev/mapper/
sudo dmsetup ls
```

Or a small script to list all open LUKS containers:

```
for dev in $(ls /dev/mapper/ | grep -v control); do
    sudo cryptsetup status "$dev"
done
```

Once I had the basic setup working, I wanted to streamline the workflow. That's where loop automation and filesystem permissions come into play.

Setting ownership and access

By default, only root can write to a freshly mounted LUKS container. To allow regular users (like myself) to work within it:

```
sudo chown -R $USER:$USER ~/securedata
```

Persistent mount options

If you plan to mount this container regularly, you can define it in `/etc/fstab`:

```
UUID=<your-uuid> <path to the directory>/securedata ext4
defaults,uid=1000,gid=1000 0 2
```

Afterward, simply run, “`sudo mount -a`”.

Automation

Eventually, I turned these commands into two simple Bash scripts, one for mounting, one for closing the container.

Mount:

```
#!/bin/bash
set -e

IMAGE="$HOME/secure_container.img"
MOUNTPOINT="$HOME/securedata"
NAME="securecontainer"

sudo losetup -fP "$IMAGE"
LOOPDEV=$(losetup -a | grep "$IMAGE" | cut -d: -f1)

sudo cryptsetup luksOpen "$LOOPDEV" "$NAME"
sudo mount /dev/mapper/"$NAME" "$MOUNTPOINT"

echo "Container mounted at $MOUNTPOINT"
```

Unmount:

```
#!/bin/bash
set -e

MOUNTPOINT="$HOME/securedata"
NAME="securecontainer"
IMAGE="$HOME/secure_container.img"

sudo umount "$MOUNTPOINT"
sudo cryptsetup luksClose "$NAME"

LOOPDEV=$(losetup -a | grep "$IMAGE" | cut -d: -f1)
sudo losetup -d "$LOOPDEV"

echo "Container closed and loop device detached."
```

Make them executable:

```
chmod +x mount-secure.sh unmount-secure.sh
```

And you've just turned your encrypted container workflow into two clean, reusable commands.

In conclusion

With LUKS, you can build a secure, flexible, and fully open-source solution that works perfectly on any Linux system, just anywhere.

This setup became part of my daily workflow for storing sensitive project data, scripts, and lab configurations. It's simple, elegant, and completely under my own control, exactly how Linux security should be.

As always, if you have any feedback, just let me know by using the contact page or leave a message on my blog post. Until next time!